

Python в примерах

М. А. Ложников

22 апреля 2019 г.

Это предварительная невыверенная версия. Читайте на свой страх и риск.

1 Циклы, операторы условного перехода и ввод/вывод

1.1 Пример 1

```
1 # Это комментарий
2
3 """
4 Это многострочный
5 комментарий
6 """
7
8 '''
9 Это тоже
10 многострочный комментарий
11 '''
12
13 i = 0      # Объявление переменной
14 n = 10     # Тип данных определяется автоматически
15
16 n = "abc"  # Тип данных может измениться при новом присваивании (теперь это строка).
17
18 print(type(n)) # Выведет информацию о типе данных переменной n
19
20 # Это обычный цикл while. Тело цикла пишется с отступом. Ширина отступа должна быть
21 # одинаковой. Обычно используют 4 пробела. Табуляцию использовать не стоит,
22 # поскольку она отображается по разному в разных редакторах. Python 3 запрещает
23 # смешивать табуляции и пробелы.
24 n = 10
25 while i < n:
26     print(i) # вывод на экран
27     i += 1
28
29 # Арифметические операторы: +, -, *, /, //, %, **, +=, -=, *=, /=, //=, %=, **=
30 # Логические операторы: <, >, <=, >=, ==, !=
31 # Отрицание: not
32 # Логическое и/или: and/or
33 # Битовые операции: |, &, ^, ~, <<, >>
```

```
34
35 # Деление / всегда вещественное. Операция // используется для целочисленного деления.
36
37 # Примеры:
38 a = 15
39 b = -5
40 c = a ** 3 # Возведение числа a в степень 3
41
42 # Логические операторы ленивые, они не вычисляются полностью. Если значение выражения
43 # уже известно, то операнды могут дальше не вычисляться.
44 s1 = "abc"
45 s2 = "def"
46 s = s1 or s2 # s = "abc" (Отлично от None и пустой строки, значит, истина.)
47 # Результат логического выражения не приводится к типу bool, значениями которого
48 # являются True и False.
49
50 # Тело оператора условного перехода тоже пишется с отступом. Слово pass пишут тогда,
51 # когда хотят оставить тело цикла, условного перехода, функции, класса пустым.
52 if not (a > 0):
53     pass
54 if a > 0 and b < 0:
55     pass
56
57 c = None # Это пустое значение, аналог NULL в C++.
58
59 # Операторы сравнения == и != можно переопределить, поэтому сравнение с None
60 # следует делать следующим образом:
61 if c is None:
62     print('c is none')
63
64 if c is not None:
65     print('c is not none')
```

1.2 Пример 2

```
1 i = 0
2 n = 10
3
4 while i < n:
5     if i < 5:
6         print('i < 5 and i = %d' % (i))
7     elif i == 5:
8         print('i = 5')
9     else:
10        print('i > 5 and i = %d' % (i))
11    i += 1
12
13
14 # Приведение типов данных
```

```
15 a = 3.5 # Для хранения используется тип данных float
16 i = int(a) # Преобразуем дробное число в целое
17
18 s = '123.45' # Строка
19 d = float(s) # Преобразуем строку в число
20 t = str(d) # Преобразуем число в строку
21 g = repr(d) # То же самое, только без округлений, способствующих читабельности
22
23 c_num = 1 + 2j # Это комплексное число
24 print(type(c_num))
25 print(c_num.real) # Действительная часть
26 print(c_num.imag) # Мнимая часть
```

1.3 Пример 3

```
1 # Так импортируются модули. Это обычные файлы на Python, которые содержат реализации
2 # функций и классов, а также объявления констант.
3 import math
4
5 # Функция input() выводит приглашение для ввода в стандартный поток вывода и
6 # считывает строку из стандартного потока ввода.
7 n = int(input('Enter n-> '))
8
9 # Такой цикл проходит по всем целым значениям от 0 до n-1.
10 for a in range(0, n):
11     print(a / math.pi) # Константа pi из модуля math --- это число Пи.
```

2 Функции

```
1 # Объявим функцию, которая складывает два своих аргумента. Тело функции пишется
2 # с отступом.
3 def sum_func(a, b):
4     return a + b
5
6 # Вызов функции
7 n = sum_func(2, 3) # n = 5
8 s = sum_func('abc', 'def') # s = 'abcdef'
9
10 # В Python при вызове функции можно передавать какому аргументу что присваивается.
11 # Это бывает полезно, например, в тех случаях, когда функция содержит много аргументов,
12 # значение которых задано по умолчанию.
13 c = sum_func(a=2, b=3) # c = 5
```

3 Строки

```
1 # Строка задаётся либо внутри двойных кавычек либо внутри одинарных
2 # Строки состоят из unicode символов.
3 a = "Hello World!"
4 b = a[4] # b = "o"
5
6 c = "34"
7 d = "56"
8 e = c + d # e = "3456" Это - конкатенация строк
9
10 g = a[0:5] # g = "Hello". Операция срезки всегда создаёт новую строку.
11 h = a[6:11] # h = "World"
```

3.1 Форматирование строк

```
1 num = 3
2 pi = 3.14159
3
4 # Так формируются строки. Формат записывается в стиле функции printf() языка C,
5 # затем после операции % идёт кортеж со значениями, которые нужно подставить.
6 s = "Num = %d and Pi = %.3f" % (num, pi)
7
8 # Ещё можно использовать метод format() класса str.
9 s = "Num = {} and Pi = {}".format(num, pi)
10
11 # Можно указать формат вывода в скобках после двоеточие.
12 s = "Num = {:05d} and Pi = {:.3f}".format(num, pi)
13
14 # Можно передать именованные аргументы:
15 s = "Num = {integer_arg} and Pi = {float_arg}".format(
16     integer_arg=num, float_arg=pi)
17
18 # Можно добавить формат вывода
19 s = "Num = {integer_arg:05d} and Pi = {float_arg:.3f}".format(
20     integer_arg=num, float_arg=pi)
21
22 # В Python 3.6 появились F-строки.
23 s = f"Num = {num} and Pi = {pi}".
24
25 # Можно с форматом
26 s = f"Num = {num: 05d} and Pi = {pi: .3f}"
```

4 Списки

```
1 """
2 Списки - это последовательности произвольных объектов. Списки создаются посредством
```

```
3 заключения элементов списка в квадратные скобки, как показано ниже:
4 """
5 names = [ "Dave", "Mark", "Ann", "Phil" ]
6
7 a = names[2] # Вернет третий элемент списка, "Ann"
8 names[0] = "Jeff" # Запишет имя "Jeff" в первый элемент списка
9
10 if "Mark" in names:
11     print("Mark is in the list")
12
13 names.append("Paula") # Добавление элемента в конец списка
14
15 names.insert(2, "Thomas") # Вставка элемента в середину списка
16
17 b = names[0:2]           # Вернет ["Jeff", "Mark"]
18 c = names[2:]           # Вернет ["Thomas", "Ann", "Phil", "Paula"]
19 names[1] = 'Jeff'       # Во второй элемент запишет имя 'Jeff'
20 names[0:2] = ['Dave', 'Mark', 'Jeff'] # Заменит первые два элемента списком справа.
21
22 a = [1, 2, 3] + [4, 5] # Создаст список [1, 2, 3, 4, 5]
23
24 # Пустой список можно создать одним из двух способов:
25 names = []           # Пустой список
26 names = list()       # Пустой список
27
28 """
29 Список может содержать объекты любого типа, включая другие списки,
30 как показано в следующем примере:
31 """
32 a = [1, "Dave", 3.14, ["Mark", 7, 9, [100, 101]], 10]
33 a[1]           # Вернет "Dave"
34 a[3][2]        # Вернет 9
35 a[3][3][1]     # Вернет 101
36
37 # Списки и цикл for
38 for i in [1, 5, 6, 8, 3, 6]:
39     print(i)
```

5 Кортежи

Кортеж — это что-то вроде структуры в языке C или `std::tuple` в C++. Иными словами, это список с фиксированным числом и типами объектов. Создаётся путём заключения группы значений в круглые скобки.

```
1 stock = ('GOOG', 100, 490.10)
2 address = ('www.python.org', 80)
3 person = (first_name, last_name, phone)
4
5 # Скобки можно опускать
```

```
6 stock = 'GOOG', 100, 490.10
7 address = 'www.python.org', 80
8 person = first_name, last_name, phone
9
10 # Ещё несколько примеров
11 a = ()          # Кортеж с нулевым количеством элементов (пустой кортеж)
12 b = (item,)     # Кортеж с одним элементом (обратите внимание на запятую в конце)
13 c = item,      # Кортеж с одним элементом (обратите внимание на запятую в конце)
14
15 # Элементы из кортежа можно извлекать с помощью индекса в квадратных скобках,
16 # однако, удобнее так:
17 name, shares, price = stock
18 host, port = address
19 first_name, last_name, phone = person
```

5.1 Пример

```
1 # Файл содержит строки вида "name,shares,price"
2 filename = "portfolio.csv"
3 portfolio = []
4
5 for line in open(filename):
6     # Преобразует строку в список, используя запятую как разделитель
7     fields = line.split(",")
8     name = fields[0]          # Извлекает и преобразует отдельные значения полей
9     shares = int(fields[1])
10    price = float(fields[2])
11    stock = (name, shares, price) # Создает кортеж (name, shares, price)
12    portfolio.append(stock)     # Добавляет в список записей
```

6 Множества

Множества используются для хранения неупорядоченных коллекций объектов.

```
1 s = set([3, 5, 9, 10]) # Создаст множество чисел
2 t = set("Hello")      # Создаст множество уникальных символов, то есть
3 # set(['H', 'e', 'l', 'o']) (здесь только один символ 'l')
4
5 if 5 in s:
6     print('5 belongs to s')
7
8 # Операции над множествами
9 a = t | s # Объединение t и s
10 b = t & s # Пересечение t и s
11 c = t - s # Разность (элементы, присутствующие в t, но отсутствующие в s)
12 d = t ^ s # Симметричная разность (элементы, присутствующие в t или в s,
13           # но не в двух множествах сразу)
14
```

```
15 # Добавление/удаление элементов
16 t.add('x')           # Добавит единственный элемент
17 s.update([10, 37, 42]) # Добавит несколько элементов в множество s
18 t.remove('H')        # Удаление элемента
```

7 Словари

Словарь это ассоциативный контейнер, содержащий объекты, индексированные ключами.

```
1 # Для создания словаря нужно заключить в фигурные скобки последовательность
2 # ключ : значение, как показано ниже
3 stock = {
4     "name": "GOOG",
5     "shares": 100,
6     "price": 490.10
7 }
8
9 # Доступ к значениям по ключу
10 name = stock["name"]
11 value = stock["shares"] * stock["price"]
12
13 # Добавление/изменение элементов
14 stock["shares"] = 75           # Изменение объекта
15 stock["date"] = "June 7, 2007" # Добавление объекта
16
17 # Пустой словарь
18 prices = {}                  # Пустой словарь
19 prices = dict()              # Пустой словарь
20
21 # Проверка наличия ключа в словаре
22 if "name" in stock:
23     print('Yes')
24 else:
25     print('No')
26
27 # Удаление элемента
28 del stock["name"]
29
30 # Получение списка ключей
31 keys = list(stock) # keys = [ "name", "shares", "price" ]
32
33 for key in stock: # Итерация по всем ключам
34     print("Key: ", key)
35
36 for val in stock.values(): # Итерация по всем значениям
37     print("Value: ", val)
38
39 for key, val in stock.items(): # Итерация по всем парам
40     print("{}: {}".format(key, val))
```

8 Чтение данных из файла

```
1 # Функция open() открывает файл. Второй необязательный параметр служит для
2 # того, чтобы указать режим: 'r' (чтение, по-умолчанию) или 'w' (запись).
3 f = open('input.txt')
4
5 line = f.readline()
6 n = int(line)
7
8 s = 0
9
10 for i in range(0, n):
11     a = float(f.readline())
12     s += a
13
14 f.close() # закрываем файл, сам собой он не закроется, даже если пропадёт
15           # из области видимости
16
17 if n > 0:
18     s /= n
19 elif n == 0:
20     print('n = 0')
21 else:
22     print('n < 0')
23
24 print('Result = ', s)
25 print('Result = %.3f' % (s))
26 print('Sequence length = %d\nResult = %.5f' % (n, s))
```

9 Аргументы командной строки

```
1 import sys
2
3 # Функция len() считает количество элементов списка. В списке sys.argv содержатся
4 # аргументы командной строки
5 if len(sys.argv) < 2:
6     print('Not enough arguments!')
7     raise SystemExit # Так генерируются исключения
8
9 # Для решения проблемы с необходимостью закрытия файлов можно использовать
10 # контекстный менеджер. Тело контекстного менеджера пишется с отступом.
11 with open(sys.argv[1]) as f:
12     lines = f.readlines() # Читаем все строки в список
13
14 # После выхода из него файл автоматически закроется.
15
16 s = 0
17 for line in lines:
```



```
18     a = float(line)
19     s += a
20
21 if len(lines) > 0:
22     s /= len(lines)
23 else:
24     print('n = 0')
25
26 print('Result = ', s)
27 print('Result = %.3f' % (s))
28 print('Sequence length = %d\nResult = %.5f' % (len(lines), s))
```

Список литературы

- [1] Бизли Д. Python. Подробный справочник. СПб: Символ-Плюс. 2010.
- [2] <https://www.python.org>
- [3] <https://www.anaconda.com/distribution>